

LAB 3

Gestione della Memoria in OS161

Programmazione di Sistema (IA-ZZ)

2024/25

Giorgio Cora

Obiettivi:

- Overview sulla gestione della memoria in OS161.
- Implementare la deallocazione della memoria e migliorare la gestione della frammentazione.

In os161, l'address space è suddiviso in 2 porzioni:

- il kernel space (0x80000000 -> 0xFFFFFFFF)
- lo user space (0x00000000 -> 0x7FFFFFFF).

Il kernel space è ulteriormente suddiviso in più parti:

- Kseg0 (0x80000000 -> 0x9FFFFFFF) -> Kernel code and data
- Kseg1 (0xA0000000 -> 0xBFFFFFFF) -> Peripheral and devices
- Kseg2 (0xC0000000 -> 0xFFFFFFFF) -> Non utilizzato

In os161 esiste già un allocatore di memoria (DUMBVM), il quale effettua allocazione di memoria in maniera contigua, senza mai rilasciarla!

Inoltre, indipendentemente dalle dimensioni di ciò che deve essere salvato in memoria, verrà sempre allocato un numero intero di pagine (1 pagina = 4096 bytes). Questo genera gravi problemi di frammentazione!

Questa operazione viene effettuata tramite 2 funzioni specifiche:

- `getppages()` -> `kern/arch/mips/vm/dumbvm.c`
- `ram_stealmem()` -> `kern/arch/mips/vm/ram.c`

```
paddr_t getppages(unsigned long npages){  
    paddr_t addr;  
    spinlock_acquire(&stealmem_lock);  
    addr = ram_stealmem(npages);  
    spinlock_release(&stealmem_lock);  
    return addr;  
}
```

Acquisisce e rilascia uno spinlock
(per gestire un corretto accesso
alla memoria ed evitare accessi
multipli indesiderati)

```
paddr_t getppages(unsigned long npages) {  
    paddr_t addr;  
    spinlock_acquire(&stealmem_lock);  
    addr = ram_stealmem(npages);  
    spinlock_release(&stealmem_lock);  
    return addr;  
}
```

Chiama la ram_stealmem
(passandogli il numero di pagine di
cui ha bisogno) per riservare dello
spazio in memoria

```
paddr_t ram_stealmem(unsigned long npages) {  
    size_t size;  
    paddr_t paddr;  
    size = npages * PAGE_SIZE;  
    if (firstpaddr + size > lastpaddr) { return 0;}  
    paddr = firstpaddr;  
    firstpaddr += size;  
    return paddr;  
}
```

Controlla se c'è spazio a
sufficienza per allocare la nuova
porzione di memoria

```
paddr_t ram_stealmem(unsigned long npages) {  
    size_t size;  
    paddr_t paddr;  
    size = npages * PAGE_SIZE;  
    if (firstpaddr + size > lastpaddr) { return 0; }  
    paddr = firstpaddr;  
    firstpaddr += size;  
    return paddr;  
}
```

Se c'è spazio a sufficienza, salva e
ritorna l'indirizzo di memoria da cui
cominciare ad allocare


```
paddr_t ram_stealmem(unsigned long npages) {  
    size_t size;  
    paddr_t paddr;  
    size = npages * PAGE_SIZE;  
    if (firstpaddr + size > lastpaddr) { return 0;}  
    paddr = firstpaddr;  
    firstpaddr += size;  
    return paddr;  
}
```

Aggiorna il puntatore globale alla memoria con la dimensione che è stata allocata

Per migliorare la gestione della memoria in os161 è necessario:

- Creare una funzione che ritorni il numero di pagine libere (funzione `getfreepages`).
- Gestire la restituzione della memoria quando viene distrutto un thread/processo (`as_destroy()` e `free_kpages()`).
- Modificare, dove necessario, le funzioni `getppages()` e `ram_stealmem()`.

L'obiettivo è quindi quello di ridurre la frammentazione generata in os161 tenendo traccia delle pagine di memoria libere o occupate.

ATTENZIONE! `Getppages()` viene chiamata sia da `kmalloc()` che `as_prepare_load()`